

Trabalho Final — Sistema de Registro de Calorias e Jejum

Visão geral

Você desenvolverá uma aplicação web full-stack para acompanhamento de consumo calórico e jejum intermitente. O sistema deverá permitir que cada usuário registre suas refeições, defina metas diárias, acompanhe ciclos de jejum e visualize seu progresso ao longo da semana.

Aviso ético: este sistema é um exercício acadêmico. Ao desenvolver, prefira uma abordagem neutra e informativa, evitando linguagem que reforce dietas restritivas ou metas agressivas. Inclua, na tela inicial ou no rodapé, um aviso de que a aplicação não substitui orientação médica ou nutricional.

Stack obrigatória

- **Framework:** Next.js 14+ (App Router)
 - **Linguagem:** TypeScript
 - **Autenticação:** Firebase Authentication **ou** Supabase Auth (à escolha)
 - **Banco de dados:** Firestore **ou** Supabase (Postgres) ou Local
 - **API:** Route Handlers do Next.js (`app/api/...`) ou Server Actions ou Local
 - **Estilização:** livre (Tailwind, shadcn/ui, CSS Modules etc.)
 - **Gráficos:** livre (Recharts, Chart.js, Tremor etc.)
 - **Deploy:** Vercel (ou equivalente público)
-

Requisitos funcionais

1. Autenticação

- Cadastro com e-mail e senha
- Login e logout
- Recuperação de senha
- Rotas protegidas: nenhum dado de usuário pode ser acessível sem autenticação
- Cada usuário deve ver **apenas** seus próprios dados

2. Registro de calorias (CRUD completo)

- Criar registro com: data/hora, descrição do alimento, calorias, tipo de refeição (café, almoço, lanche, jantar, ceia)
- Listar registros com filtro por data
- Editar qualquer registro existente
- Excluir registro (com confirmação)

3. Meta calórica

- Definir meta diária de calorias
- Editar meta a qualquer momento
- Exibir consumo atual versus meta no dashboard (ex.: barra de progresso)

4. Registro de jejum

- Iniciar um jejum (registra horário de início)
- Encerrar o jejum em andamento (registra horário de fim e calcula duração)
- Selecionar tipo planejado: 16:8, 18:6, 20:4, 24h ou personalizado
- Apenas um jejum ativo por vez

5. Históricos

- Histórico de jejuns

6. Resumo semanal (gráfico)

- Gráfico de calorias consumidas por dia nos últimos 7 dias, com linha de referência da meta
- Gráfico (ou indicador equivalente) de horas de jejum por dia nos últimos 7 dias
- Indicadores agregados: média diária de calorias, total de jejuns concluídos na semana, tempo médio de jejum

Requisitos não funcionais

- **Responsividade:** funcional em mobile e desktop
- **Validação:** validar entradas no client e no server (sugestão: Zod)
- **Estados de UI:** tratar loading, erro e estado vazio em todas as telas
- **Segurança:** regras do Firestore / Row Level Security do Supabase devidamente configuradas
- **Variáveis de ambiente:** chaves sensíveis nunca commitadas; `.env.example` no repositório

- **Acessibilidade básica:** uso correto de labels, contraste adequado, navegação por teclado nos formulários principais
-

Entregáveis

1. **Repositório público no GitHub** com:
 - Código-fonte
 - **README .md** contendo: descrição, stack, instruções de setup local, variáveis de ambiente necessárias, link da aplicação em produção, screenshots das telas principais
 - Histórico de commits coerente (não vale 1 commit único)
 2. **Aplicação publicada** com URL acessível
 3. **Vídeo demonstrativo** de 3 a 5 minutos mostrando o uso real da aplicação (upload no Drive, YouTube unlisted ou Loom)
-

Critérios de avaliação

Critério	Peso
Funcionalidades implementadas conforme requisitos	35%
Qualidade do código (organização, tipagem, separação de responsabilidades)	20%
Modelagem de dados e segurança (regras de acesso, validações)	15%
UI/UX (clareza, responsividade, feedback ao usuário)	15%
Documentação e entrega (README, deploy funcional, vídeo)	10%
Commits e processo de desenvolvimento	5%

Bônus (até 1,0 ponto na nota final):

- Modo escuro funcional
- Exportação dos dados em CSV ou JSON
- PWA instalável
- Testes automatizados (unitários ou e2e)
- Notificação ao final do jejum planejado
- Importação de alimentos a partir de uma API pública (ex.: OpenFoodFacts)

Regras

- Trabalho **individual** (ou em dupla)
 - Uso de IA generativa é **permitido**, mas o aluno deve compreender e saber explicar todo o código entregue. Haverá arguição.
 - Plágio entre trabalhos zera a nota dos envolvidos.
 - Atrasos: desconto de 10% por dia, até no máximo 3 dias.
-